

Miller-Rabin 素性测试实现和测试

202400460046 邱泽辉

2026 年 5 月 8 日

首先根据课本给出伪代码实现 Miller-Rabin 算法，随后使用 Python 和 C++ 结合 NTL 库分别实现，并测试了 2048bit 和 4096bit 的随机数以及素数的判定效率。

Algorithm 1 Miller-Rabin 算法

```
1: Input: 整数  $n$ 
2: Output: 判断  $n$  是否为素数
3: 将  $n - 1$  表示为  $n - 1 = 2^l m$ , 其中  $m$  是奇数
4: 随机选取整数  $a$ , 满足  $1 < a < n - 1$ 
5: 计算  $b \leftarrow a^m \bmod n$ 
6: if  $b \equiv 1 \pmod{n}$  then
7:   return “ $n$  是素数”
8: end if
9: for  $i = 0$  to  $l - 1$  do
10:  if  $b \equiv -1 \pmod{n}$  then
11:    return “ $n$  是素数”
12:  else
13:     $b \leftarrow b^2 \bmod n$ 
14:  end if
15: end for
16: return “ $n$  是合数”
```

当然这里可以做一些改进，比如大于 2 的偶数肯定是合数等，基于此我们给出 python 实现如下：

```
1  import random
2  from Crypto.Util.number import *
3  import time
4  def Miller_Rabin(n):
```

```
5     if n == 2 or n == 3:
6         print(f"{n} 是素数")
7         return 1
8     if n % 2 == 0:
9         print(f"{n} 是合数,并且是偶数")
10        return 0
11
12    l = 0
13    m = n - 1
14    while(m%2==0):
15        m //= 2
16        l += 1
17    a = random.randint(2,n-2)
18    b = pow(a,m,n)
19    if b == 1:
20        print(f"{n} 是素数")
21        return 1
22    for i in range(0,l):
23        if b == n-1:
24            print(f"{n} 是素数")
25            return 1
26        else:
27            b = pow(b,2,n)
28    print(f"{n} 是合数")
29    return 0
30
31 def test(N):
32     start = time.time()
33     Miller_Rabin(N)
34     end = time.time()
35     return (end - start)*1000
36
37 N = random.randint(2**2047,2**2048-1)
38 P = getPrime(2048)
39 N1 = random.randint(2**4095,2**4096-1)
40 P1 = getPrime(4096)
41 print(f"测试2048bit的数")
42 print(f"{test(N)} ms")
43 print(f"测试2048bit素数")
44 print(f"{test(P)} ms")
45 print(f"测试4096bit的数")
```

```
43     print(f"{test(N1)} ms")
44     print(f"测试4096bit素数")
45     print(f"{test(P1)} ms")
```

C++ 结合 NTL 实现如下:

```
1  #include <NTL/ZZ.h>
2  #include <iostream>
3  #include <chrono>
4  using namespace std;
5  using namespace NTL;
6  bool MillerRabin(ZZ n) {
7      ZZ m = n - 1;
8      int l = 0;
9      while (m % 2 == 0) {
10         m /= 2;
11         l++;
12     }
13     ZZ a = RandomBnd(n-3) + 2;
14     ZZ b = PowerMod(a, m, n);
15     if (b == 1) {
16         //printf("素数\n");
17         return true;
18     }
19     for(int i = 0; i < l; i++) {
20         if (b == n - 1) {
21             //printf("素数\n");
22             return true;
23         }
24         b = PowerMod(b, 2, n);
25     }
26     //printf("合数\n");
27     return false;
28 }
29
30 void test(ZZ n) {
31     auto start = chrono::high_resolution_clock::now();
32     bool result = MillerRabin(n);
33     auto end = chrono::high_resolution_clock::now();
```

```
34     auto duration = chrono::duration_cast<chrono::milliseconds>(end - start);
35     cout << "Miller-Rabin算法判定" << n << "为" << (result ? "素数" : "合数") <<
        "耗时" << duration.count() << "ms" << endl;
36 }
37 int main() {
38     printf("测试2048bit的素数\n");
39     ZZ P = GenPrime_ZZ(2048);
40     test(P);
41     printf("测试2048bit的数\n");
42     ZZ N = RandomBits_ZZ(2048);
43     test(N);
44     printf("测试4096bit的素数\n");
45     ZZ P1 = GenPrime_ZZ(4096);
46     test(P1);
47     printf("测试4096bit的数\n");
48     ZZ N1 = RandomBits_ZZ(4096);
49     test(N1);
50     return 0;
51 }
```

分析

从测试结果可以看出，Miller-Rabin 算法在 Python 和 C++ 中的性能差异较大。C++ 的实现明显更快，尤其是在处理大位数的整数时。这主要是因为 C++ 的编译器优化和更高效的内存管理。此外，NTL 库在 C++ 中提供了高效的大整数运算，这也显著提升了性能。

随机生成一个整数，它是合数的概率较高，因此在测试中采用随机数来测试合数

而素数则是用库函数生成的，但问题是，库函数好像本身就是基于多轮 Miller-Rabin 算法实现的，所以生成的素数在测试中自然会被判定为素数，这似乎有一些循环论证的感觉，不过多轮 Miller-Rabin 算法就可以极大降低误判的概率，可以认为这个素数是可信的

实验中发现，python 生成 4096bit 的素数需要的时间比较久，主要时间都在等待素数的生成，而 C++ 则没有这个问题。

实现	位数	判定	时间 (ms)
Python	2048	合数	24.934
Python	2048	素数	26.014
Python	4096	合数	195.112
Python	4096	素数	193.670
C++	2048	素数	2
C++	2048	合数	2
C++	4096	素数	13
C++	4096	合数	17

表 1: Miller-Rabin 性能对比

附录：测试结果

1. Python：2048 bit 合数测试

```
2826832642394794627053567953755461246769186637572783522121853553429733637141310557858702961965141514
1406613190529467363894466194398415124037289102078662932819142054866262939967443790101314267461124254
7910185971265421488852989247354429607476142728890816478037685362409832763842911530374136719037956629
9537922985105241383541321528705713133938511156221383447835860438887130136267270011989911674177348502
1140291862743977271780281517899092258312157342808125760483927117966139848626748406257902869944488789
3621986537411939265086690585532737263333169866270837124446718195684278970070584014640480987177349979
23958713756492183
```

判定结果：合数；耗时：24.934 ms。

2. Python：2048 bit 素数测试

```
3113993577511641842278716365940462079560143173542467952573891355189426564047935740490618169210959469
0497789493985029075432689292294324916419477675518186199814387510730377801104494176159653085777404674
4065530323028027515745654619892039247741657239335625561680545292741484666071103453102574312649760926
0623449150371967898599922354092563455367851309723097159852253750603724325736516718049222510931463853
3795679596087937743076098720596223594252200775441054867423745784932885668269386752327389635713779900
5734735093086585629245009326124097179606717212548298973855785384603746310286977088176903387767206429
37584466366110381
```

判定结果：素数；耗时：26.014 ms。

3. Python：4096 bit 合数测试

```
8925526967951989020587380780685839335269610320727776465489633114309295394206133906450739833156839880
7980449120087649809070698606823110370521022934187375086231789301070796876424779787722642116298856748
6406305667258346332069457139510485302051304075655173246975569476052964504777053580433269965041284085
```

```
3805792819913805655604814303129863439661123204344047543754018986309887960511361711904986214146155213
0791762494782179933773200301638024928146847714478684921091833366615374652709803682690371659395226102
0408621449485341933243051629982174165968769639595965718036320037071768142697357354965261460861196528
7223023003755910978490421165213812253105762005328421379632156564824590983542402417486721759971614382
5928551949141271048639056853580383657130074584770659817556883743807820385430351262449094275959759927
5008461647421331459663157860415411413905019566128475035621155117404684672942255425205874355160235927
9442535646058640752397717105668668624868942470122666421428496109129705261338548672271623841419912584
189089168414247361298853449887773386776128775623117832482795214748857780172317656701516273287097066
9751878619477870733919834693067706684060871512812664548099261552177108548144232056211040650346827469
760831396056370545499170121529992
```

判定结果：合数；耗时：195.112 ms。

4. Python：4096 bit 素数测试

```
1019825969191054889395772952531320169932775379799580317685372045362121104111009966109926520195095962
3171202555209371416459906213914187459512111259196562445595963646450763713670930399289677310718680405
4951140773172194174909241231178450420453695566101933111857695148457341350184436904203495724495592773
1923216366612983959342046967765279134210240828083225227324308952997277879918310378644604278856552642
9851668615053644030345088330106724575319311758085542607539119409112916371728141548399860476141864530
4080698826547898818166323147722716231113874375749490289215145684900456214634897771052543933479795840
1790412425904258624340188765039224515144080230934722301430122226857685130180567563971987176813688529
8097756762414289423427933827423375562438499201089234712540377825901529271363366687433880074371133119
3338634210775003456877541561205297813232475949287553515806893852747459942259708980662764475932505586
5419680640525008321748866966009119918058817383369934087807470369103901597227184753455983658379508710
0565763978960306620259314603809565303980723346118283987733369163487367845810690443919972578790835882
7009500191271733620388162321659827818988150458756199490892183789932853867973442064992323806700311015
2733051538574798220756537410172737
```

判定结果：素数；耗时：193.670 ms。

5. C++：2048 bit 素数测试

```
2039029083347791175449514962424947719798676116289494954682753086635997009705903277086620842337463474
3888041881685333472744882736849523372201187094688815688744794032494461716588087570549607825649176514
8968384426512563039472726340492804517943471268164855250177555060674686344889826161739474366883114679
7148595033667795837191393250297242715686665732209357431033575613694088727963350082994629366158467596
0789627499726429436475907787349800057788566991147984984918958869514892670803536520048971075358718350
1914485766025984205203442565506595030663831646466890672256726582839621451571037198854466319685214304
58843869648899149
```

判定结果：素数；耗时：2 ms。

6. C++：2048 bit 合数测试

```
2326533765438464268159917020650811528851161254918679864745827473410726468987628423035042810415135386
5590768235670249522627185391144843333552219807616584009945691715482836451379321031500121130234022163
2627009758018024658250875618101298277412282509483046053172297093449182375286932141595448477052520330
0418899327788167015916114825607923713174543517996064582507129117352825428583915039520348462070896290
```

```
5582809527472264597679956374246798446645919064864715278055145308016187016706043337203717645867427084
7028509776698497088072047486917023853121522103608766798261241514643730817399814650713765013310676574
41862083815373568
```

判定结果：合数；耗时：2 ms。

7. C++：4096 bit 素数测试

```
8904653566152935026435485022020367092625337191432224016910242963969686170611197503506612098661387240
5879904760839283742454310291971076311510332595596656979272321387822285710991692306244266324066737997
9918805049477421422899039493315985225988290239966055210085617240009552675230877001958717303701677054
0093424687956771422974926079297412504705899804102718985881322281986760837487210808471460969793576510
1132296544921933864636119572373530844779179525265645505079684606692872036864305443592689234650319927
3740798078424174761731331751858027462168436016142195463910202615662660239381131910260991624602769200
1948953244071696947637445142172682806062795063080209826018836303303161336590415272261341027902768898
5708318169733168738504340420622382506478139815232429234323635546138665107192550712232679655035688645
5311876755496839499428704995541953969853875056558423708209731444855309652217413574740928108688663841
6724177197141607617465907429937070984202014620314841791046670208894015448115530331498359309715884343
6620335144502636692788579706965809519525858389092704685997005070703522755577705840391848843887385782
594600521433860435449095801980277507960365069675312951394159298225179180470338199393004413775046125
507674184200534603875154816871791
```

判定结果：素数；耗时：13 ms。

8. C++：4096 bit 合数测试

```
8855600210937835133056877034991371019895213310999185081313262064695699713347119617727345824034435219
5237399496190900714703159592551210478309964746553613846609974132348224251252669992352890801541020911
7916226992307227088335739642680503922310744546364390712646981791745874518417723448947338724762120818
9347412582605496030486060580164076469369973057009862590295373608381151956443442437852808029943204748
7421680826816290632371262368388785199429161509091932329457882940726884290072810848575124983992319516
8181352324088519535334893513928184077120873045684059217903877339054187281073505159282962477364527041
3880216164817563752953716073347843883277803313311560418899249015138006634873036546191538327552196846
2229314273066642733371557297733793239053291353544305696096645527537673269463859702041726865241978137
7250850928245199412431609772843183094245459258827818182589540875051121860232978328331765460325896439
4954644975248918016362053642344040636751372372012921259960800961367312764251317891527163578179603083
7330136813456205072329907643613035580377165753191552854491015679969714721971078614240603990121161635
2720360994245146930618182771540742183458991045391758790874519387681228591429699101837476787074993470
617508116790215823910398568586434
```

判定结果：合数；耗时：17 ms。